Teaching Tiny Robots Teamwork Together

William Qian

Ms. Vidya Mannem

Dr. Claire Walton



Background

Reinforcement Learning

- A field that has grown increasingly popular over the recent years
- Involves training a robot to do a certain task through positive and negative feedback
- The robot essentially is guided by random actions and will learn behaviors to get the more reward and the least punishment
- The robot has a neural network, much like an artificial intelligence like ChatGPT and such
- Training robots through reinforcement learning does require an extremely powerful computer

```python
class CommandsCfg:
    """Command specifications for the MDP."""

    base_velocity = mdp.UniformVelocityCommandCfg(
        asset_name="robot",
        resampling_time_range=(5.0, 5.0),
        rel_standing_envs=0.0,
        rel_heading_envs=1.0,
        heading_command=False,
        heading_control_stiffness=0.5,
        debug_vis=False,
        ranges=mdp.UniformVelocityCommandCfg.Ranges(
            lin_vel_x=(10.0, 10.0), lin_vel_y=(0.0, 0.0), ang_vel_z=(1.0, 1.0), heading=None #(0.0, 0.0)
        ),
    )
```

Curriculum and commands

- Commands
  - The goals for the training, what the E-puck trains to be.
- Curriculum
  - Starting simple, then progressively giving more complicated commands
    - Very useful for training a robot from scratch like what I am doing.
- Termination criteria
  - What causes the training to terminate
    - Can be set to many different triggers such as an out of bounds error, or termination after a set period of time.

```python
@configclass
class RewardsCfg:
    """Reward terms for the MDP."""

    terminated = RewTerm(func=mdp.is_terminated, weight=-100.0)

    Epuck_track_lin_vel_xy_yaw_frame_exp = RewTerm(
        func=mdp.epuck_track_lin_vel_xy_yaw_frame_exp,
        weight=1.0,
        params={"command_name": "base_velocity", "asset_cfg": SceneEntityCfg("robot"),
        },
    )

    Epuck_track_ang_vel_z_exp = RewTerm(
        func=mdp.epuck_track_ang_vel_z_world_exp, weight=0.2, params={"command_name": "base_velocity", "std": 1.0}
    )
```

Values for training

- Rewards/Punishments
  - These values will be unique to whatever we are training
    - Some common rewards are
      - Alive Reward: Encourage the agent to stay alive for as long as possible
      - Terminating Reward: Similarly penalize the agent for terminating
    - Some rewards I anticipate will be used for the E-puck preliminary testing are
      - Cart revolutions in the given period yields rewards
      - Punishment for straying from the radius of the circle
    - The actual rewards
      - Staying withing a set radius
      - Driving at a target velocity

Plan

Original plan

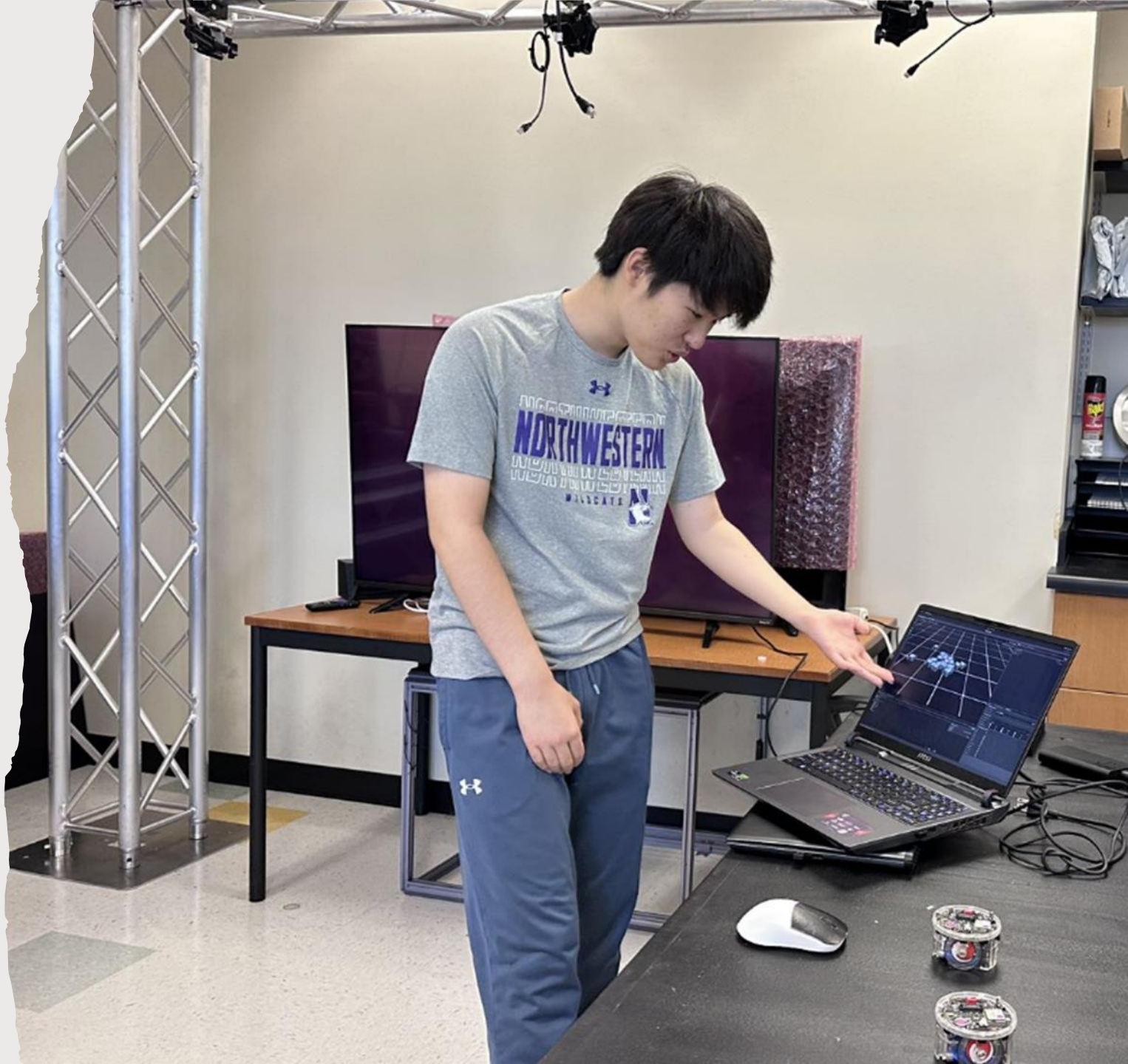
- Planned to have a 4 E-puck system work together to drive in various patterns and navigate through an environment
- Would train through Isaac Lab
- Will sense the surroundings and respond accordingly

What actually happened

- Had both a single E-puck working to do a variety of circles in simulation and a system of 4 training to do a circle
- As of right now, the lab is still working on a way to translate my trained system to the real world.

The Process

- Set up Isaac Lab
- Create an environment
  - Create USD
  - Code config
- Start running
  - Tweak the code and fix bugs
  - Change values until the optimal outcome is achieved



Setting up IsaacLab

- IsaacLab was a lot more complicated to set up than previously anticipated due to a variety of factors
- Omniverse depreciation
  - This means Isaac-sim can not be run at the latest updated version and will not be able to run IsaacLab inside
- Complicated installation process
  - Installing Isaac-sim 4.5.0 and then IsaacLab inside of it is much more complicated than going online then pressing a download button
  - Will need to download IsaacLab through console commands and a variety of online repositories. As well as installing the necessary dependencies.

How to create an environment

- Environment creation first needs a script to import the necessary parameters and stuff for the environment to function.
  - This can be done through any python IDE (I’m using visual studio)
  - This also needs a model that is the main focus of the training
    - I am using a model of the E-puck 2 found online from the official website and modified to work in IsaacLab
  - You need to configure values for the model and path the right directory to it
  - An environment configuration is also necessary

Struggles and Challenges

- Incapable hardware
- Windows-Linux divide
- Flying E-pucks
- Inherent training times
- Learning curve

Incapable hardware

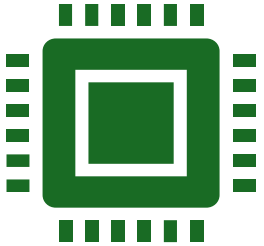


I had anticipated this long before starting the project, so I had contingencies.

It indeed did come to pass that my laptop was not powerful enough to run the simulations required for my project as at least a Nvidia Geforce RTX 3080 was required to even run Isaaclab.

I had some very powerful computers for public use at the Lab I was working in

I was however too stubborn to learn L inux until very late into the project, where I borrowed a RTX 4090 from the lab



New computer

I went from a 3-year-old RTX 3050 to a brand new RTX 4070. This change allowed me to run a simulation on my personal computer as well.

Flying E-pucks

- One problem I encountered on my windows device was the flying E-puck issue.
- After a while of training, the E-pucks would learn how to fly.
  - Like no joke they would take off and zoom through the air.
- I tried a variety of fixes for it, but as I was quite inexperienced at the time, I couldn't find a solution.
- I then asked my mentor and a few UTSA students familiar with this and they could not find a solution.
- In the final weeks of the project I relented and learned how to use a linux computer to train my E-puck system and it worked perfectly.
- Determined to make a windows system work in the final week of the project, I was able to successfully create an environment that would train properly and not cause flying E-pucks in my windows computer with my newfound experience.

Windows Linux divide

Windows code

```
usd_path=r"C:\Users\Larry\IsaacLab\source\isaaclab_tasks\isaaclab_tasks\manager_based\epuck\epuck(1).usd",
```

Context

- Windows and Linux are two vastly different computer operating systems.
- I grew up mainly familiar with windows, while the people working at the lab are experts with linux.
  - This caused a bit of a problem where the discrepancies between linux and windows began to cause issues on my windows operating system
  - The students couldn’t help much with debugging as they are not familiar with how code works in windows
  - I had to learn linux from scratch

Linux code

```
usd_path=f"/home/rl_robotics_10/IsaacLab/source/isaaclab_tasks/isaaclab_tasks/manager_based/epuck/epuck(1).usd"
```

Inherent Training times

- Reinforcement learning is very advanced technology and thus requires an insane amount of processing power.
  - However, even with the best computers, the process is still very slow.
- This was more of an annoyance than an actual serious problem, but it did make things go a lot slower
  - Every time I ran the program; I had to wait for quite a while, depending on how many iterations I gave it
  - Every time it failed, I had to change some values and repeat the process over again

Learning curve

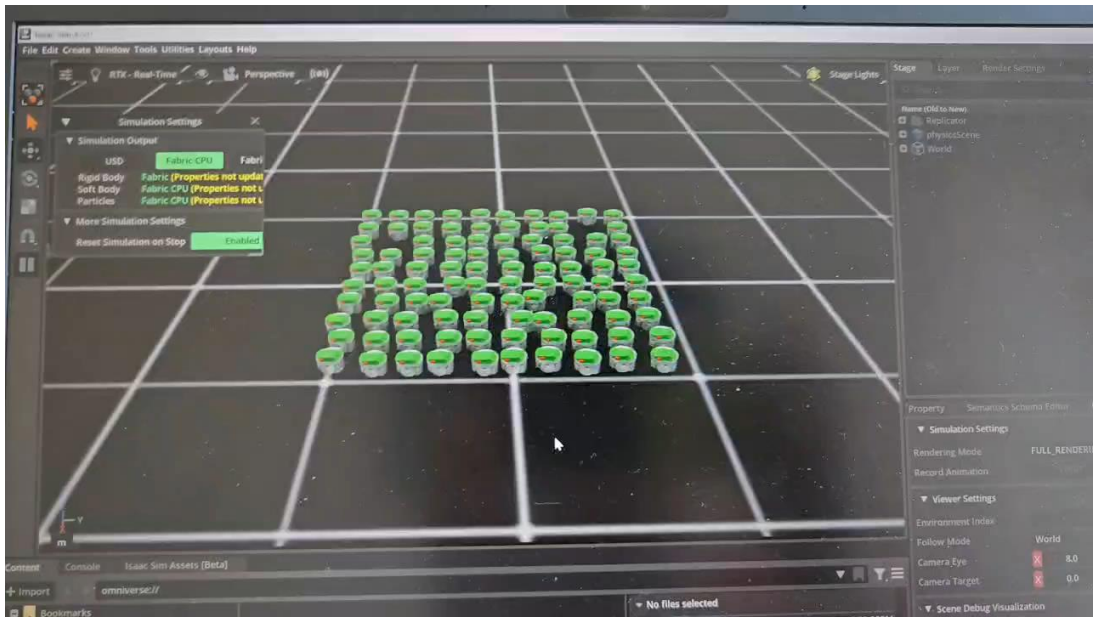
- Reinforcement learning to train robotic systems is a rather new discovery
  - And certainly not what you would expect a high schooler to know how to use
- Since it is so advanced, it is very unusual to attempt such a project at this age
  - Most of the people I was working with had multiple years of experience and were PhD students
  - I attempted at least a year’s worth of work in 3 months

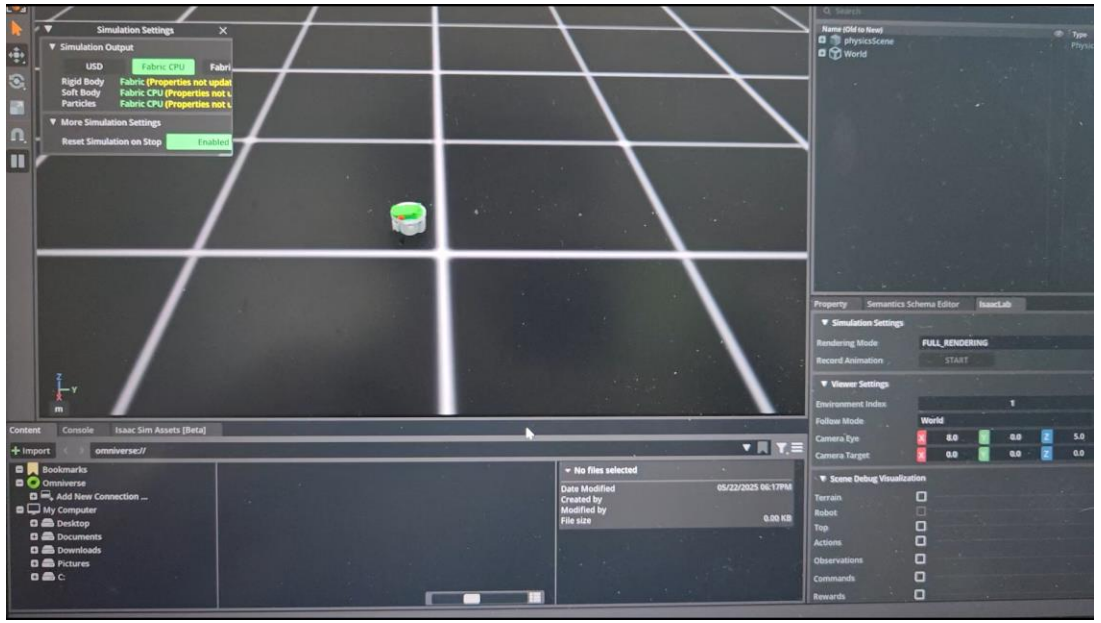
Important events/Milestones

- Downloaded Isaac-sim 4.5.0 through a very complicated process
- Downloaded IsaacLab into Isaac-sim
- Learned to crawl directories and how the file manager works.
- Finished adjusting the USD file found online
- Worked with UTSA students to create configuration files
- Ran a successful Linux simulation
- Ran a successful simulation of multiple E-pucks at the same time
- Ran a successful windows simulation
- Finished training an agent for both the 4 E-puck system and the individual E-puck

Demos

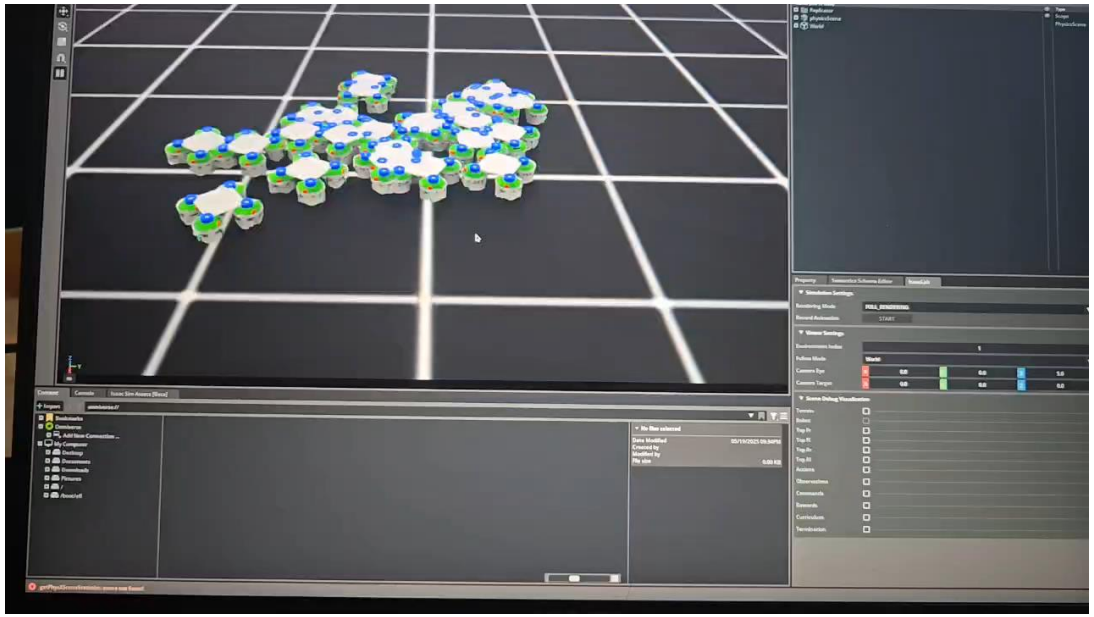
Single E-puck



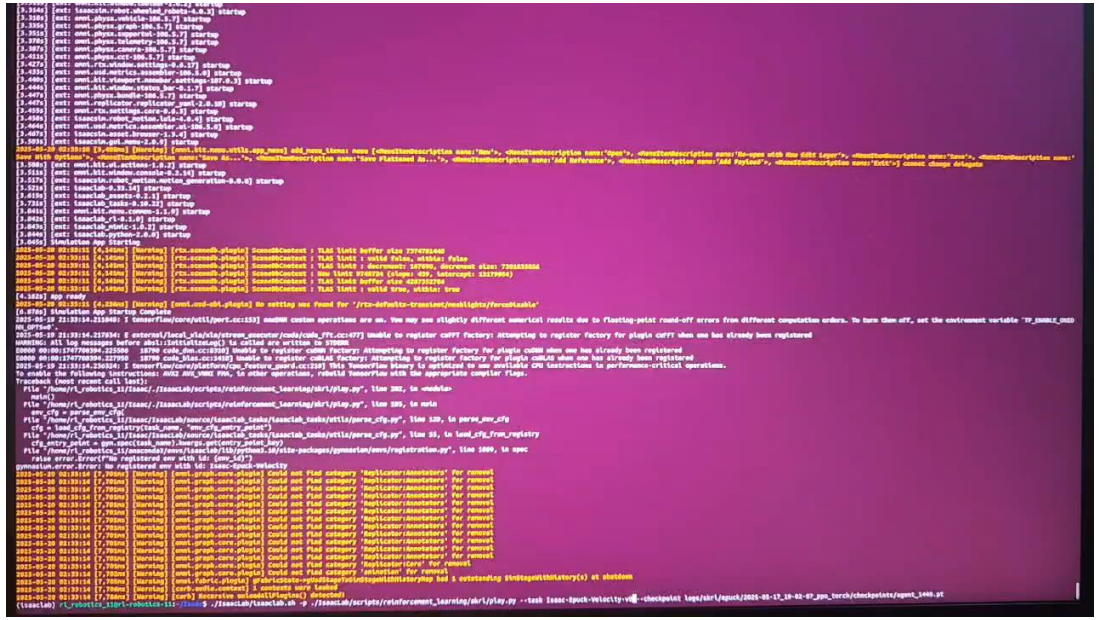


4 E-puck system

Training



Demonstration



Results

The E-puck was able to train successfully on both a linux and a windows system

The E-puck system of 4 was also able to successfully train

The result will be reported in a conference paper coauthored by Dr. Walton and her students.

It is being implemented into the real robots by Jack for his PhD dissertation.

Conclusion

- I learned a lot through the research project
  - How to install programs the nonconventional way through console
  - How to create a python environment to run reinforcement learning
  - How to think and solve problems
- The project was successful as I came into the project knowing very little and walked away with a whole lot of experience
  - Additionally, this was a great help to the lab who will be building on my research in the future as they did not have a way to train the E-pucks until now.

Thank you!

- Professor Claire Walton, PhD, UTSA
- PhD student, Jack Han, UTSA
- PhD student, Sergio Montufar, UTSA
- Undergraduate student Alex Roynon, UTSA

